UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

PROGRAMACION III
BACK END

Unidad N°3:
Algoritmos

Material de Estudio
2° Año – 3° Cuatrimestre

CC BY ND NC

Índice

1. ¿Qué es un algoritmo? 2

1.1 Clasificación de Algoritmos 3

1.1.1 Clasificación según el tipo de problema que resuelven 3

1.1.2 Clasificación según su forma de operar 4

1.1.3 Clasificación según su eficiencia esperada 6

2. Complejidad de los Algoritmos 6

2.1 ¿Qué es la complejidad de un algoritmo? 7

2.2 ¿Cómo medimos qué tan complejo es un algoritmo? 7

2.4 Complejidad Temporal 9

2.5 Notación Big O (O Grande) 11

2.6 Tipos de Eficiencias Comunes 12

A. O(1) – Constante 12

B. O(n) – Lineal 13

C. O(n^2) – Cuadrática 13

D. O(log n) – Logarítmica (Ej: Búsqueda binaria) 14

2.7 Ejemplos de Algoritmos Mal Diseñados 15

3. Analizando algoritmos Típicos: Búsqueda Lineal vs Búsqueda Binaria 20

3.1 Búsqueda Lineal 20

3.2 Búsqueda Binaria 21

3.3 Comparación de Eficiencia 23

4. Análisis del Algoritmo de Fibonacci - Algoritmos recursividad vs Algoritmos iterativos 24

4.1 ¿Qué es la sucesión de Fibonacci? 24

4.2 Resolviendo Fibonacci de forma recursiva 26

4.3 Complejidad de la solución recursiva 27

4.4 Alternativa: Programación Dinámica 27

4.6 Conclusión 28

5. Estrategias de Resolución de Problemas 28

1. ¿Qué es un algoritmo?

Antes de adentrarnos en temas como eficiencia, análisis y estrategias de diseño, es importante detenernos un momento y repasar qué entendemos por **algoritmo**, por qué son tan importantes en programación, y qué propiedades básicas deben cumplir para ser considerados correctamente definidos.

Un algoritmo es **un conjunto de pasos ordenados y finitos** que permiten resolver un problema o realizar una tarea específica.

En programación, un algoritmo es la base para escribir código: primero pensamos **qué pasos se deben seguir para llegar a una solución**, y luego los convertimos en instrucciones en un lenguaje de programación.

Por ejemplo, un algoritmo puede indicar cómo ordenar una lista de números, cómo buscar un valor específico en una base de datos o cómo calcular el promedio de notas de un estudiante.

Lo más importante de un algoritmo es que:

1. **Debe ser claro y preciso**: cada paso tiene que estar bien definido.
2. **Debe tener un fin**: no puede ejecutarse para siempre.
3. **Debe ser efectivo**: cada paso debe poder realizarse en un tiempo razonable y con los recursos disponibles.

Ejemplo cotidiano: Preparar un mate

Podemos pensar que hacer un mate es un algoritmo. Veamos los pasos:

1. Calentar agua sin que hierva (idealmente a unos 75-80°C).
2. Colocar yerba en el mate (llenar unos 2/3 del recipiente).
3. Inclinar el mate para que la yerba se acumule hacia un lado.
4. Verter un poco de agua tibia en la parte más baja (opcional, para hidratar).
5. Colocar la bombilla en ese hueco.
6. Verter agua caliente en la parte donde está la bombilla.
7. Tomar el mate y repetir el paso 6 hasta que la yerba pierda sabor.

Ejemplo de programación: Calcular el promedio de 3 números

Pensamos el problema y lo resolvemos con pasos ordenados:

1. Leer tres números ingresados por el usuario.
2. Sumar los tres números.
3. Dividir el resultado entre 3.
4. Mostrar el promedio en pantalla.

Y su versión en pseudocódigo podría ser:

```
Inicio
    Leer numero1
    Leer numero2
    Leer numero3
    suma <- numero1 + numero2 + numero3
    promedio <- suma / 3
    Mostrar promedio
Fin
```

1.1 Clasificación de Algoritmos

Los algoritmos pueden organizarse en distintas **categorías** según cómo funcionan o qué tipo de problema resuelven. Esta clasificación no solo nos ayuda a entenderlos mejor, sino que también nos permite **elegir la estrategia adecuada** cuando enfrentamos un nuevo desafío de programación.

A continuación, veremos las clasificaciones más comunes y útiles al momento de estudiar o diseñar algoritmos.

Simplemente mencionaremos las distintas categorías y a través de toda la unidad profundizaremos en algunas.

1.1.1 Clasificación según el tipo de problema que resuelven

Algoritmos de búsqueda

Estos algoritmos se utilizan para **encontrar un elemento** dentro de una estructura de datos, como un arreglo o una lista.

Ejemplos:

- Búsqueda lineal (`O(n)`)
- Búsqueda binaria (`O(log n)`, requiere orden)

Algoritmos de ordenamiento

Se utilizan para **reorganizar elementos** en una estructura de datos según algún criterio (por ejemplo, de menor a mayor).

Ejemplos:

- Bubble sort (burbuja) – simple pero poco eficiente
- Merge sort – más eficiente, basado en divide y conquistar
- Quick sort – muy usado en la práctica

Algoritmos de optimización

Buscan la **mejor solución posible** (óptima) entre muchas alternativas, de acuerdo a una cierta medida.

Ejemplos:

- Problema de la mochila (knapsack)
- Caminos mínimos (como Dijkstra)
- Algoritmos de asignación de tareas

Algoritmos de recorrido

Se usan para explorar estructuras de datos, como listas enlazadas, árboles o grafos.

Ejemplos:

- Recorridos en profundidad (DFS)
- Recorridos en anchura (BFS)
- Recorridos en árboles binarios (inorden, preorden, postorden)

1.1.2 Clasificación según su forma de operar

Algoritmos iterativos

Requieren repetir ciertas instrucciones usando **estructuras de control como bucles** (`for, while`). Son muy comunes y suelen ser más eficientes en consumo de memoria que su versión recursiva.

Ejemplo:

```
int sumar(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) {
        total += arr[i];
    }
    return total;
}
```

Algoritmos recursivos

Se **llaman a sí mismos** para resolver subproblemas más pequeños. Son elegantes y naturales para ciertos problemas, como el cálculo de Fibonacci o recorridos de árboles.

Ejemplo:

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

Importante: La recursión puede ser más costosa en memoria si no se optimiza.

Algoritmos deterministas vs. no deterministas

- **Deterministas:** Siempre producen el **mismo resultado** para una misma entrada.
- **No deterministas (o probabilísticos):** Pueden generar **diferentes resultados** aunque la entrada sea la misma, o depender del azar o decisiones aleatorias.

Ejemplo determinista: La búsqueda binaria siempre sigue el mismo camino para una entrada dada.

Ejemplo no determinista: Algoritmos genéticos o algunos métodos de machine learning pueden variar su salida en cada ejecución.

1.1.3 Clasificación según su eficiencia esperada

Otra forma común de clasificar algoritmos es observando **cómo crece su tiempo de ejecución** a medida que aumenta el tamaño de los datos de entrada. Esto se describe usando **notación Big O**.

Ejemplo comparativo simple:

- **Búsqueda lineal:** O(n)
- **Búsqueda binaria:** O(log n)
- **Bubble sort:** $O(n^2)$
- **Merge sort:** O(n log n)

2. Complejidad de los Algoritmos

Hasta ahora hemos aprendido a escribir algoritmos correctos: reciben datos, hacen cálculos y devuelven el resultado esperado. Pero en programación no alcanza con que un algoritmo "**funcione**": también nos interesa saber **qué tan bien funciona**.

Por ejemplo:

- ¿Tarda mucho o poco en terminar?
- ¿Usa mucha memoria o recursos de la computadora?
- ¿Es más rápido que otras formas de resolver el mismo problema?

Aunque dos algoritmos hagan exactamente lo mismo, uno puede ser más eficiente que otro. **Evaluar el rendimiento de un algoritmo es parte clave del trabajo de un programador**. Algunos resuelven un problema en pocos pasos, otros tardan mucho más. Algunos consumen poca memoria, otros pueden agotar los recursos de la computadora. Por eso, **evaluar el rendimiento de un algoritmo** es fundamental.

En esta sección vamos a aprender a responder preguntas como:

- ¿Cuánto tarda un algoritmo en ejecutarse?
- ¿Qué tanto depende del tamaño de los datos que recibe? ¿El algoritmo se vuelve más lento?
- ¿Qué tan "eficiente" es una solución en comparación con otra?
- ¿Podemos estimar cuánto tiempo va a tardar sin ejecutarlo?

En todos los casos hablamos de eficiencia, pero ¿Cómo medimos la eficiencia?

Para responder a esa pregunta, vamos a introducir herramientas y conceptos que nos permitan **medir y analizar la eficiencia de los algoritmos**, vamos a ver la **notación Big O,** que nos ayuda a **describir el comportamiento de un algoritmo cuando la cantidad de datos crece**. También veremos ejemplos concretos que nos ayudarán a desarrollar una mirada crítica sobre nuestras propias soluciones.

2.1 ¿Qué es la complejidad de un algoritmo?

La **complejidad** de un algoritmo se refiere a la cantidad de **recursos computacionales** que necesita para resolver un problema. Los dos recursos más importantes que se analizan son:

- **Tiempo:** Cuánto tarda el algoritmo en ejecutarse y cómo evoluciona este tiempo acorde a cuanto crece el tamaño de tus entradas.
- **Espacio:** Cuánta memoria necesita durante su ejecución, es decir, si se usan variables innecesarias o si se almacena información que no hace falta o que se almacena un dato de manera repetida.

2.2 ¿Cómo medimos qué tan complejo es un algoritmo?

La forma más común de medir la complejidad de un algoritmo es estimando **cómo crecen sus recursos necesarios (tiempo o espacio)** en función del tamaño del problema de entrada, que se suele denotar con la letra **n**.

Por ejemplo, si tenemos un algoritmo que recorre una lista de tamaño n, el tiempo que tarda depende de cuántos elementos tenga la lista. Si la lista tiene 10 elementos, puede tardar poco; si tiene 10.000, puede tardar mucho más.

Un algoritmo que funciona bien con 10 elementos puede volverse muy lento con 1000 o más. Por eso, es importante pensar no solo en que el algoritmo funcione, sino también en cómo escala a medida que el tamaño del problema crece. Un algoritmo **poco eficiente** puede funcionar bien para casos pequeños, pero volverse **impracticable** cuando el tamaño de la entrada crece.

Esta relación se expresa usando una notación matemática que describe el comportamiento del algoritmo en su "**peor caso**", "**mejor caso**" o "**caso promedio**".

Mejor caso

Es la situación más favorable posible para el algoritmo, la entrada tiene una forma que permite resolver el problema muy rápido, haciendo pocas operaciones. En otras palabras, el algoritmo se ejecuta en el menor tiempo posible.

Ejemplo

Supongamos que queremos buscar el número `5` en esta lista:

```
[5, 8, 12, 20, 25, 30]
```

Usando búsqueda lineal recorremos la lista uno por uno.

- En este caso, el número `5` está **en la primera posición**.
- Solo se necesita **una comparación** para encontrarlo.

Peor caso

Es la situación más desfavorable, el algoritmo tiene que trabajar más porque la entrada lo obliga a hacer muchas operaciones. El algoritmo tarda lo máximo posible.

Ejemplo

Buscamos el número `40` en esta lista:

```
[5, 8, 12, 20, 25, 30]
```

- El número **no está** en la lista.
- El algoritmo tiene que recorrer **toda la lista completa** y verificar uno por uno.

Caso promedio

Es el comportamiento habitual del algoritmo con entradas al azar o sin una estructura especial. Representa lo que suele pasar en la práctica.

Ejemplo

Imaginemos que el número que buscamos **sí está en la lista**, pero su posición **es aleatoria**.

En promedio, lo vamos a encontrar:

- En la **mitad de la lista**, si todos los elementos son igualmente probables.
- Entonces, hacemos aproximadamente `n / 2` comparaciones.

2.4 Complejidad Temporal

¿Qué es? La **complejidad temporal** es una forma de analizar **cuánto tiempo tarda un algoritmo en ejecutarse**, Pero en lugar de medir ese tiempo con un reloj (que depende de la computadora) lo medimos en función **de la cantidad de operaciones elementales que realiza: comparaciones, asignaciones, ciclos, etc**..

Como las computadoras tienen velocidades diferentes, **no se mide en segundos**, sino en **la cantidad de pasos básicos** que realiza el algoritmo. Esto permite comparar algoritmos de forma objetiva, sin importar en qué máquina se ejecutan.

¿Cómo se mide?

- Se analiza el número de **iteraciones**, **comparaciones**, **asignaciones** u otras operaciones clave. ¿Cuántas veces se repite un bucle? ¿Cuántas comparaciones se hacen? ¿Cuántas veces se llama a una función?
- Se considera cómo cambia ese número cuando el tamaño de entrada **n** crece.

Ejemplo 1 – Revisar una lista de clientes

Imaginemos que tenemos una empresa que envía correos electrónicos a sus clientes para ofrecerles una promoción. La empresa guarda una lista de clientes con sus nombres y correos electrónicos.

Queremos hacer un algoritmo que recorra esa lista y envíe un mail personalizado a cada cliente.

```
for (int i = 0; i < clientes.length; i++) {
    enviarCorreo(clientes[i]);
}
```

En este caso:

- La función enviarCorreo() se ejecuta una vez por cada cliente.
- Si hay 10 clientes, se hacen 10 envíos.
- Si hay 1000 clientes, se hacen 1000 envíos.

La cantidad de operaciones crece proporcionalmente a la cantidad de clientes (n).

Complejidad temporal: O(n) (Se llama “lineal” porque el tiempo crece en línea recta con la cantidad de elementos).

Ejemplo 2 – Comparar pares de productos

Ahora imaginemos un sistema que analiza una lista de productos para detectar productos muy similares y evitar duplicados en el catálogo. Para hacer eso, el sistema compara cada producto con todos los demás, evaluando si el nombre, precio y categoría son muy parecidos.

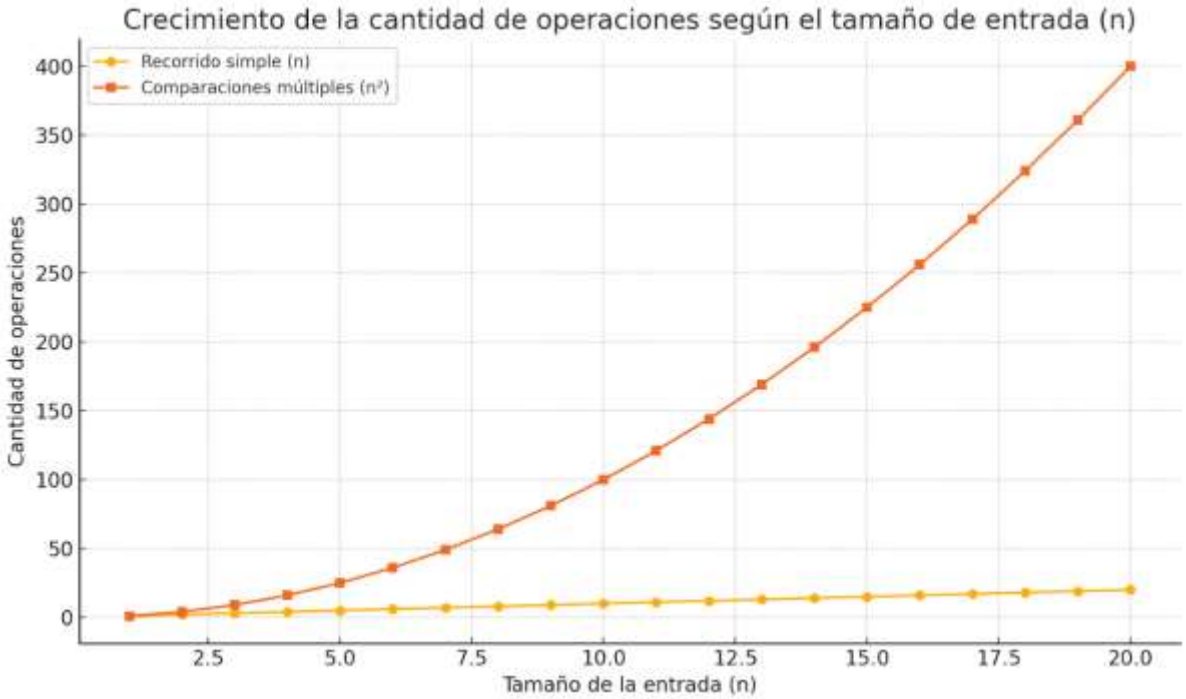
```
for (int i = 0; i < productos.length; i++) {
    for (int j = 0; j < productos.length; j++) {
        if (i != j && sonSimilares(productos[i], productos[j])) {
            marcarComoPosibleDuplicado(productos[i], productos[j]);
        }
    }
}
```

En este caso:

- Si hay 10 productos, se hacen aproximadamente 100 comparaciones (10 × 10).
- Si hay 100 productos, se hacen 10.000 comparaciones (100 x 100).
- Si hay 1000 productos, se hacen 1.000.000 de comparaciones (1000 x 1000).

El tiempo crece mucho más rápido a medida que aumenta la cantidad de productos.

Complejidad temporal: $O(n^2)$ (Se llama “cuadrática” porque el tiempo crece como el cuadrado de la cantidad de elementos).



En el gráfico se muestra claramente cómo crece la cantidad de operaciones en ambos casos:

- La línea amarilla representa un algoritmo lineal (ejemplo 1), donde las operaciones crecen en forma proporcional a n.
- La línea naranja muestra un algoritmo cuadrático (ejemplo 2), donde el número de operaciones crece mucho más rápido a medida que n aumenta.

2.5 Notación Big O (O Grande)

La **notación Big O** (se lee "O de") es una herramienta matemática que usamos para expresar la **tasa de crecimiento**, es decir, **cuánto crece el tiempo de ejecución de un algoritmo a medida que crece el tamaño de la entrada**, que solemos llamar n.

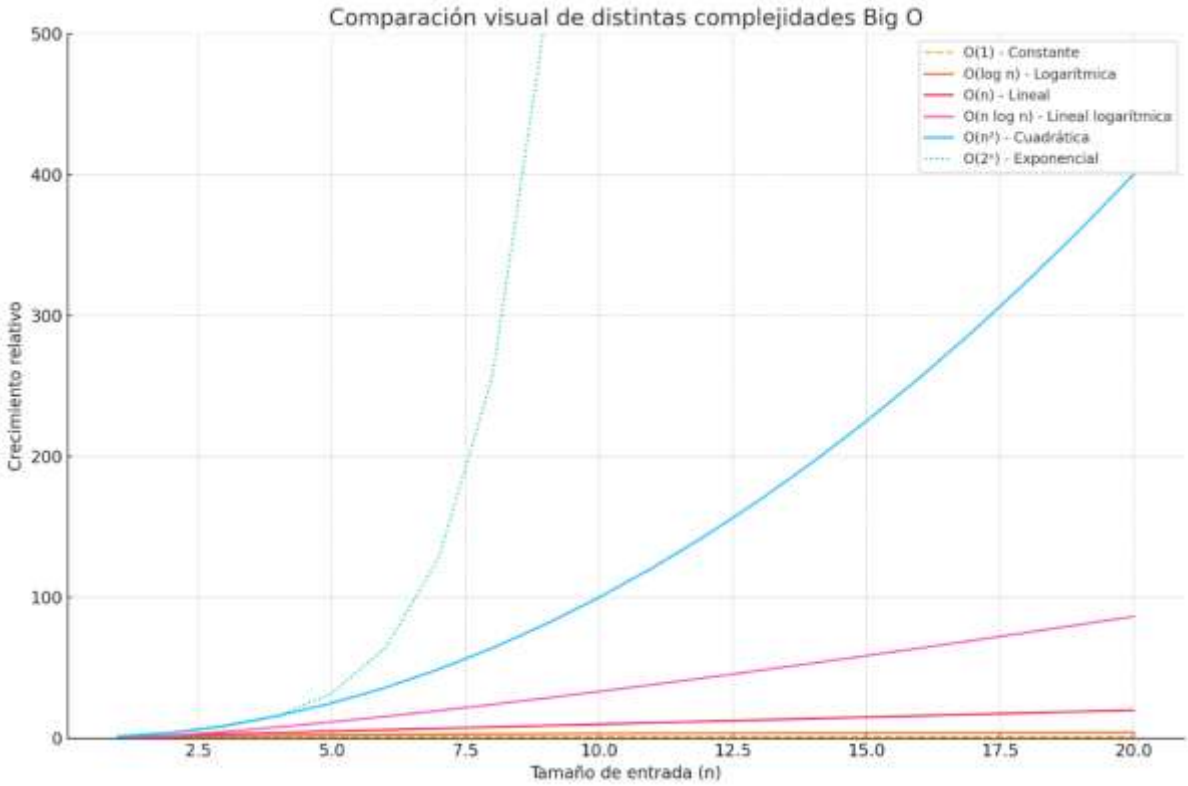
Se enfoca en el **comportamiento del algoritmo cuando n tiende a ser muy grande**, es decir, lo que llamamos el **caso asintótico (cuando el tamaño de la entrada tiende al infinito.)**.

¿Qué indica la notación O?

Indica el **límite superior**, es decir, una estimación del peor crecimiento posible del algoritmo, ignorando constantes y términos de menor orden. No nos dice exactamente cuánto tarda un algoritmo, sino cómo escala.

Por ejemplo, si un algoritmo es O(n), eso significa que si duplicamos el tamaño de la entrada, el tiempo de ejecución también se duplicará (aproximadamente). Pero si es $O(n^2)$, duplicar la entrada puede cuadruplicar el tiempo.

Expresión Big O	Nombre	Descripción rápida
O(1)	Tiempo constante	No depende del tamaño de entrada. Siempre tarda lo mismo, sin importar el tamaño de entrada.
O(log n)	Tiempo logarítmico	Crece lentamente al aumentar n (ej: búsqueda binaria).
O(n)	Tiempo lineal	Crece proporcionalmente al tamaño de la entrada.
O(n log n)	Tiempo lineal logarítmico	Crece un poco más rápido que O(n). Común en algoritmos eficientes de ordenamiento.
$O(n^2)$	Tiempo cuadrático	Muy costoso para entradas grandes. Ejemplo: comparación de todos contra todos.
$O(2^n)$, O(n!)	Tiempo exponencial o factorial	Impracticable para entradas grandes.



Las curvas muestran claramente cómo algunos algoritmos crecen de forma razonable (como O(n), O(log n)) mientras que otros (como $O(n^2)$ y especialmente $O(2^n)$) se vuelven inviables muy rápido.

2.6 Tipos de Eficiencias Comunes

A. O(1) – Constante

Una operación tiene **complejidad constante** cuando **su tiempo de ejecución no depende del tamaño de los datos**. Siempre tarda lo mismo, con un solo paso o acceso.

Ejemplo: Atención al primer cliente en fila preferencial

Imagina que tenés un sistema en una clínica médica que gestiona turnos. Hay una fila preferencial (por ejemplo, personas mayores o con discapacidad) y cada vez que llega el médico, el sistema debe decirle quién es el primer paciente preferencial para atender.

El sistema guarda esta fila en un arreglo, donde el primero en entrar está en la primera posición (índice 0).

```
String obtenerPrimerPaciente(String[] filaPreferencial) {
    return filaPreferencial[0];
}
```

No importa si hay 1 persona en la fila o 10 o 1000, el tiempo de ejecución es el mismo.

B. O(n) – Lineal

Un algoritmo tiene **complejidad lineal** cuando su tiempo de ejecución **crece en proporción directa al tamaño de los datos**. Si los datos se duplican, el tiempo también (más o menos) se duplica.

Ejemplo: "Buscar el número de ficha de un paciente en la sala de espera"

Imaginá que estás en una clínica que no usa pantallas ni turnos digitales. Hay una enfermera con una **lista de papel** con los nombres y números de ficha de los pacientes que están en la sala de espera.

Si la enfermera necesita buscar la ficha de una persona específica, tiene que leer la lista desde el principio, nombre por nombre, hasta encontrarla.

```
int buscarFicha(String[] listaPacientes, String nombreBuscado) {
    for (int i = 0; i < listaPacientes.length; i++) {
        if (listaPacientes[i].equals(nombreBuscado)) return i;
    }
    return -1; // Si no está
}
```

Si hay 10 pacientes, se harían hasta 10 comparaciones, si hay 100 pacientes, hasta 100. Es decir que a medida que la lista crece, el tiempo de búsqueda también crece al **mismo ritmo**.

C. O(n^2) – Cuadrática

Un algoritmo tiene **complejidad cuadrática** cuando **por cada elemento del conjunto, se realiza un recorrido completo sobre todo el conjunto otra vez**.

Ejemplo: "Organizar entrevistas entre alumnos"

Supongamos que en una jornada vocacional de una escuela, se quiere organizar entrevistas rápidas entre los alumnos para que **cada uno hable con todos los demás** al menos una vez.

La consigna es: **cada alumno se tiene que entrevistar con todos los otros**.

Si hay 10 alumnos, se arman entrevistas así:

- Alumno 1 habla con el 2, 3, 4, ..., 10
- Alumno 2 habla con el 1, 3, 4, ..., 10

Cada combinación representa una entrevista única.

```
void organizarEntrevistas(String[] alumnos) {
    for (int i = 0; i < alumnos.length; i++) {
        for (int j = 0; j < alumnos.length; j++) {
            if (i != j) {
                System.out.println(alumnos[i] + " entrevista a " + alumnos[j]);
            }
        }
    }
}
```

¿Qué pasa con más alumnos?

- Con 5 alumnos: 5 × 5 = 25 combinaciones (menos 5 de sí mismos = 20 entrevistas)
- Con 100 alumnos: 100 × 100 = 10.000 combinaciones (menos 100 de sí mismos = 9.900 entrevistas)

D. O(log n) – Logarítmica (Ej: Búsqueda binaria)

La complejidad logarítmica es una de las más eficientes.

Antes de avanzar, entendamos que es un logaritmo: Un logaritmo responde a la pregunta, ¿A qué número tengo que elevar la base para obtener un resultado?

Entonces ***$Log_b(x) = y$*** significa ***b elevado a la y da como resultado x***.

Ejemplo simple: ***$log_2(8)=3$***

Un algoritmo con esta complejidad **reduce el tamaño del problema a la mitad en cada paso**.

Eso significa que **aunque los datos crezcan muchísimo, el número de operaciones crece muy poco**.

```
int busquedaBinaria(int[] arr, int objetivo) {
    int inicio = 0, fin = arr.length - 1;
    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;
        if (arr[medio] == objetivo) return medio;
        else if (arr[medio] < objetivo) inicio = medio + 1;
        else fin = medio - 1;
    }
    return -1;
}
```

En cada paso se reduce la búsqueda a la mitad. Muy eficiente en listas ordenadas.

¿Cómo lo relacionamos con el logaritmo?

Supongamos que tenés una lista de tamaño n. **¿Cuántas veces podés dividirla a la mitad hasta que quede 1 solo elemento?** Eso es, ***$log_2(n)$***

Ejemplos:

- Si `n = 8`, necesitás 3 pasos: 8 → 4 → 2 → 1
- Si `n = 16`, necesitás 4 pasos: 16 → 8 → 4 → 2 → 1

Esos pasos son el **logaritmo en base 2 de** `n`: ***log2(8)=3, log2(16)=4***

2.7 Ejemplos de Algoritmos Mal Diseñados

2.7.1 No aprovechar estructuras de datos adecuadas

Ejemplo: búsqueda de un usuario por email

```
String buscarEmail(String[] emailUsuarios, String email) {
    for (String e : emailUsuarios) {
        if (e.equals(email)) return e;
    }
    return null;
}
```

¿Qué hace este código?

- Recorre uno por uno el arreglo `emailUsuarios`.
- Compara cada email con el email que se quiere buscar.
- Si encuentra uno que coincide, lo devuelve. Si llega al final sin encontrarlo, devuelve `null`.

Problemas con esta implementación

- **Búsqueda lineal en cada llamada:** Este código tiene complejidad **O(n)**, es decir, en el peor caso debe recorrer **todos los elementos de la lista**. Si hay 10.000 usuarios registrados, y el email buscado está al final o no está, el algoritmo hará 10.000 comparaciones.
- **Escala mal con listas grandes:** Este enfoque **funciona bien con listas pequeñas**, pero a medida que la cantidad de usuarios crece, **el rendimiento se degrada rápidamente**.

Solución: usar estructuras de datos adecuadas

En este caso, lo más eficiente es usar una estructura de datos que permita **acceso directo** en lugar de recorrido completo.

En Java, una estructura ideal para esto es un Set (o incluso un Map, si se asocian valores al email).

```
Set<String> usuarios = new HashSet<>(Arrays.asList(listaUsuarios));
if (usuarios.contains(email)) {
    System.out.println("Usuario encontrado.");
}
```

Aclaración: Al crear ese set implícitamente se está recorriendo todo el array, por lo que no es eficiente, pero nos referimos a que si originalmente la información viene almacenada en un Set entonces su acceso si va a ser constante O(1).

¿Qué mejora esto?

1. Velocidad de búsqueda

- `HashSet.contains()` tiene complejidad O(1) en promedio.
- Esto significa que la **búsqueda es casi inmediata**, sin importar si hay 100 o 100.000 usuarios.

2. Código más claro

- No necesitamos recorrer manualmente la lista.
- La intención del código ("ver si un usuario está registrado") es más directa y legible.

3. Escalabilidad

- Esta solución escala mucho mejor.
- Podes realizar millones de búsquedas por segundo sin que se degrade el rendimiento.

2.7.2 Recorridos innecesarios

Ejemplo: filtrar clientes activos de una lista

Imaginá que estás desarrollando un sistema para el área de atención al cliente de una empresa.

La empresa tiene una lista de todos sus clientes, pero solo quiere **mostrar los nombres de aquellos que están activos** (por ejemplo, tienen una suscripción vigente).

```
List<Cliente> activos = new ArrayList<>();

// Primer recorrido: filtrar los activos
for (Cliente c : clientes) {
    if (c.activo) {
        activos.add(c);
    }
}
// Segundo recorrido: mostrar los activos
for (Cliente c : activos) {
    System.out.println(c.nombre);
}
```

Problema:

1. **Dos recorridos completos de listas**:
   - Primero recorremos toda la lista de `clientes` para armar la lista de `activos`.
   - Después volvemos a recorrer la lista `activos` para mostrar los resultados.
   - Si hay 100.000 clientes, eso son hasta **200.000 iteraciones**.
2. **Uso innecesario de memoria**:
   - Estamos creando una nueva lista (`activos`) solo para mostrar los nombres.
   - Si no necesitamos guardar los clientes activos para usarlos más adelante, **esta lista es innecesaria**.

Solución:

Podemos simplemente recorrer una única vez

```
for (Cliente c : clientes) {
    if (c.activo) {
        System.out.println(c.nombre);
    }
}
```

2.7.3 Uso innecesario de bucles anidados

Ejemplo: verificar si hay nombres repetidos en una lista

Imagina que estás desarrollando un sistema para inscribir personas en un taller. Alguien te pide validar si hay **nombres duplicados** en la lista de inscriptos, ya que no se deberían anotar dos personas con el mismo nombre (por ejemplo, por error de doble carga).

```
for (int i = 0; i < nombres.length; i++) {
    for (int j = 0; j < nombres.length; j++) {
        if (i != j && nombres[i].equals(nombres[j])) {
            System.out.println("Nombre duplicado: " + nombres[i]);
        }
    }
}
```

Problemas:

1. Complejidad cuadrática: O(n²)

- Por cada nombre en la lista, se comparan todos los demás.
- Si hay 10.000 nombres, se hacen 100 millones de comparaciones. ¡Ineficiente!

2. Comparaciones duplicadas

- El par "Ana" vs "Lucía" y "Lucía" vs "Ana" se comparan dos veces.
- No solo se gasta tiempo, sino que no aporta nada nuevo.

3. Resultados repetidos

- Si "Pedro" aparece 3 veces, se imprimirá varias veces como duplicado.
- Esto ensucia la salida y puede confundir al usuario.

Solución:

Podemos usar un `Hashes` para **detectar duplicados en tiempo O(n)** y asegurarnos de que **cada nombre duplicado se imprima una sola vez**.

```
Set<String> nombresVistos = new HashSet<>();
Set<String> duplicados = new HashSet<>();

for (String nombre : nombres) {
    if (!nombresVistos.add(nombre)) {
        duplicados.add(nombre);
    }
}

for (String nombre : duplicados) {
    System.out.println("Nombre duplicado: " + nombre);
}
```

Ahora que mejoramos nuestro algoritmo, la pregunta que debemos hacernos es… ¿Podemos mejorar aún más este algoritmo? La respuesta es depende…

En términos de complejidad Big-O, no se puede hacer mejor que O(n) si necesitas detectar duplicados en una colección de `n` elementos.

¿Por qué no se puede mejorar de O(n)?

Porque **necesitás ver todos los elementos al menos una vez** para saber si hay algún duplicado. Saltarte algún elemento podría hacer que te pierdas un duplicado, por lo tanto, **O(n)** es lo **mejor que se puede lograr** en términos de tiempo.

Entonces… ¿Se puede optimizar algo más? La respuesta es SI, pero ya no en tiempo, sino en otros aspectos:

Opción 1 - Menor uso de memoria

Si quieres optimizar espacio (por ejemplo, no usar `Set` extra para duplicados), podrías usar solo uno:

```
Set<String> vistos = new HashSet<>();

for (String nombre : nombres) {
    if (!vistos.add(nombre)) {
        System.out.println("Nombre duplicado: " + nombre);
    }
}
```

- No guarda los duplicados, solo los imprime cuando los encuentra.
- Usa un solo `HashSet` → menos memoria.
- Sigue siendo O(n) en tiempo y espacio.

Opción 2 - Ordenar primero (con costo O(n log n))

Si por alguna razón no podés usar estructuras como HashSet, podés ordenar primero y luego comparar:

```
Arrays.sort(nombres); // O(n log n)

for (int i = 1; i < nombres.length; i++) {
    if (nombres[i].equals(nombres[i - 1])) {
        System.out.println("Nombre duplicado: " + nombres[i]);
    }
}
```

- Complejidad total: O(n log n) (por el sort)
- No necesita HashSet
- Buena opción si los datos ya vienen ordenados o si el espacio importa más que el tiempo.

Conclusión

Alternativa	Complejidad	Usa memoria extra	Notas
Doble for	**$O(n^2)$**	No	Muy ineficiente
HashSet + Set duplicados	**O(n)**	Sí	Eficiente, clara
HashSet solo (sin duplicados)	**O(n)**	Sí (pero menos)	Ahorra memoria
Ordenar y comparar	**O(n log n)**	No	Eficiente si no se puede usar Set

Consejos generales para el diseño de buenos algoritmos

1. **Pensar antes de escribir código**: ¿puedo resolver esto con un solo recorrido? ¿Qué estructura me ayuda más?
2. **Elegir la estructura de datos adecuada**: listas, sets, mapas, colas, pilas... cada una tiene su lugar.
3. **Analizar la complejidad temporal y espacial**: no basta con que funcione, también tiene que ser eficiente.
4. **Buscar la simplicidad**: muchas veces, el mejor algoritmo es el más simple de entender y explicar.

3. Analizando algoritmos Típicos: Búsqueda Lineal vs Búsqueda Binaria

En esta sección vamos a analizar **dos algoritmos clásicos de búsqueda**: la **búsqueda lineal** y la **búsqueda binaria**. Ambos se utilizan para encontrar un elemento dentro de una estructura de datos (como un arreglo), pero su eficiencia es muy diferente.

3.1 Búsqueda Lineal

La búsqueda lineal, también llamada búsqueda secuencial, es uno de los algoritmos más simples para encontrar un elemento dentro de una colección de datos, como un arreglo o una lista.

¿Cómo funciona?

La búsqueda lineal recorre los elementos **uno por uno** desde el comienzo hasta el final. En cada paso, compara el elemento actual con el valor buscado. Si lo encuentra, devuelve su posición; si llega al final sin encontrarlo, indica que el valor no está presente.

Ejemplo en Java:

```
int buscarLineal(int[] arr, int objetivo) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == objetivo) return i;
    }
    return -1; // No se encontró
}
```

Características:

- No necesita que el arreglo esté ordenado.
- Es simple de implementar.
- Sirve para cualquier tipo de datos.
- No escala bien, su rendimiento empeora linealmente con el tamaño de la entrada.

Complejidad:

- **Mejor caso:** O(1) (si está al principio).
- **Peor caso**: O(n) (si está al final o no está).
- **Caso promedio**: O(n). (hay que recorrer la mitad del arreglo, en promedio.)

3.2 Búsqueda Binaria

La **búsqueda binaria** es un algoritmo de búsqueda **mucho más eficiente** que la búsqueda lineal, especialmente en listas grandes. Sin embargo, tiene un requisito importante: **el arreglo debe estar ordenado previamente**. Si los datos no están ordenados, la búsqueda binaria no funciona correctamente.

¿Cómo funciona?

La idea principal es **reducir el espacio de búsqueda a la mitad en cada paso**. En lugar de recorrer todos los elementos, el algoritmo decide en qué mitad continuar basándose en comparaciones con el elemento del medio.

Pasos del algoritmo:

1. Se toma el índice del medio del arreglo.
2. Se compara ese valor con el que estamos buscando.
3. Si es igual, se encontró el valor.
4. Si el valor buscado es menor que el del medio, se repite el proceso en la **mitad izquierda** del arreglo.
5. Si es mayor, se repite en la **mitad derecha**.

El proceso continúa hasta encontrar el valor o hasta que no quede más por buscar.

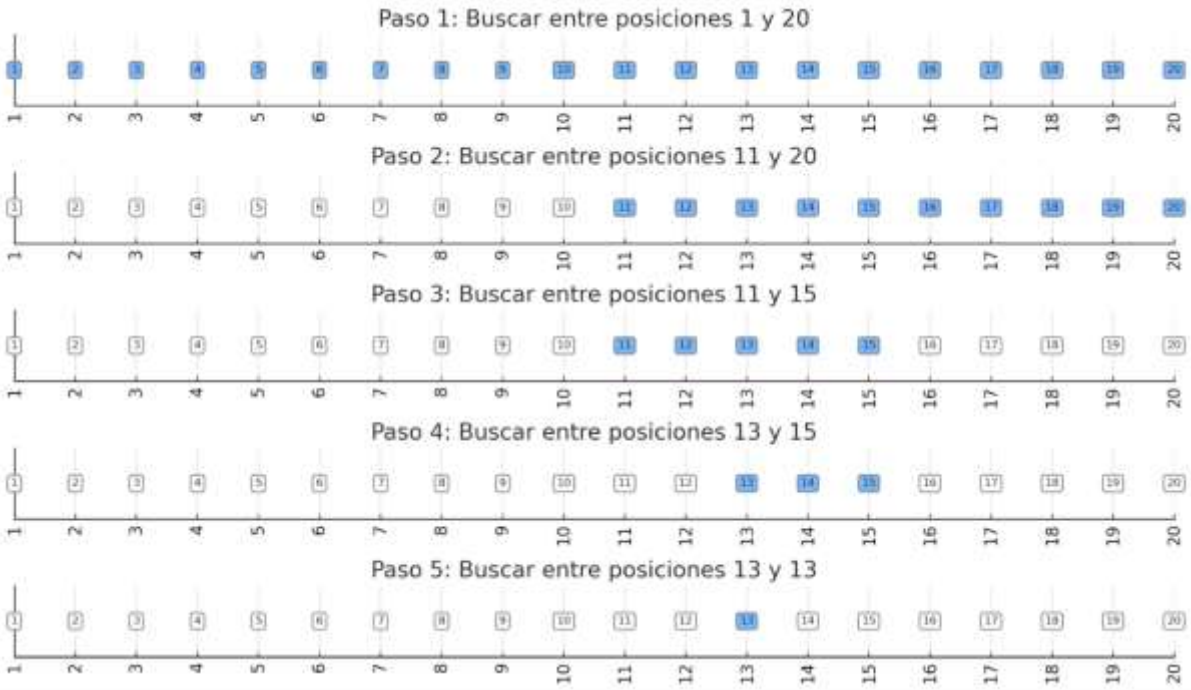
Cada paso **descarta la mitad de los elementos**, lo que hace que sea muy rápido incluso con listas grandes.

Ejemplo gráfico:

Supongamos que tenemos una lista ordenada de valores del 1 al 20 y queremos encontrar el número 13. La secuencia a seguir sería la siguiente:

Paso a paso:

1. **Paso 1:** El algoritmo comienza considerando **toda la lista** (del 1 al 20).
2. **Paso 2:** Compara el valor en la mitad (posición 10, valor 11). Como 13 es mayor, descarta la mitad izquierda y busca entre los valores **11 al 20**.
3. **Paso 3:** Compara el nuevo valor central (posición 15, valor 16). Como 13 es menor, descarta la mitad derecha y continúa buscando entre **11 al 15**.
4. **Paso 4:** Compara el nuevo centro (posición 13, valor 14). Como 13 es menor, vuelve a reducir el rango: ahora busca entre **13 al 15**.
5. **Paso 5:** Finalmente, encuentra el valor 13 en la posición 12. La búsqueda termina exitosamente.



Ejemplo en Java:

```
int busquedaBinaria(int[] arr, int objetivo) {
    int inicio = 0, fin = arr.length - 1;
    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;
        if (arr[medio] == objetivo) return medio;
        else if (arr[medio] < objetivo) inicio = medio + 1;
        else fin = medio - 1;
    }
    return -1;
}
```

Características:

- Solo funciona si el arreglo está ordenado.
- Muy rápida incluso con listas grandes.
- Divide el espacio de búsqueda a la mitad en cada paso.

Complejidad:

- **Mejor caso**: O(1) (si justo está en el medio).
- **Peor caso**: O(log n). (va dividiendo por la mitad hasta encontrarlo)
- **Caso promedio**: O(log n). (muy eficiente)

3.3 Comparación de Eficiencia

Supongamos que tenemos un arreglo de tamaño `n = 1.000.000` y queremos encontrar un número:

Algoritmo	Comparaciones en el peor caso
Búsqueda lineal	hasta 1.000.000
Búsqueda binaria	hasta $\log_2(1.000.000) \approx 20$

Como se puede ver, **la búsqueda binaria es muchísimo más eficiente en arreglos grandes**.. Mientras la búsqueda lineal puede requerir revisar cada uno de los elementos (un millón de elementos), la binaria lo hace en alrededor de 20 pasos.

4. Análisis del Algoritmo de Fibonacci - Algoritmos recursividad vs Algoritmos iterativos

4.1 ¿Qué es la sucesión de Fibonacci?

La **sucesión de Fibonacci** debe su nombre a **Leonardo de Pisa**, un matemático italiano del siglo XIII más conocido como *Fibonacci*. Fue uno de los primeros europeos en estudiar matemáticas provenientes del mundo árabe e hindú, y jugó un papel fundamental en la difusión del sistema decimal (los números arábigos) en Europa.

Fibonacci presentó esta famosa secuencia en su libro **"Liber Abaci" (El libro del ábaco)**, publicado en 1202. El libro no estaba centrado en la sucesión en sí, sino en enseñar el sistema de numeración indo-arábigo y resolver problemas prácticos de comercio y cálculo.

La secuencia aparece en el contexto de un **problema sobre reproducción de conejos**. El enunciado es más o menos así:

"¿Cuántos pares de conejos se pueden generar en un año a partir de una sola pareja, si cada mes cada pareja produce una nueva pareja que se vuelve fértil al segundo mes?"

A partir de este planteo, Fibonacci desarrolló un modelo de crecimiento en el que los números generados seguían la ahora famosa secuencia:

```
1, 1, 2, 3, 5, 8, 13, 21, 34, ...
```

Aunque a simple vista parezca solo una curiosidad matemática, la sucesión de Fibonacci **aparece en muchos contextos distintos**, tanto dentro como fuera del mundo de la informática:

- **En la naturaleza:** Se observan patrones similares en la disposición de hojas, ramas, pétalos de flores, o la espiral de las conchas.
- **En finanzas:** Algunos modelos de predicción de mercados usan proporciones basadas en Fibonacci.
- **En arte y arquitectura:** Las proporciones de la llamada "razón áurea", que está relacionada con Fibonacci, se aplican desde hace siglos.

Formalmente, se puede definir así:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$, para $n \geq 2$

¿Qué significa cada parte?

- F(0) = 0

Esto indica que el **primer valor de la sucesión (posición 0)** es 0. Es el **caso base** más simple. Se establece como punto de partida.

Es como decir: si no hay nada, la cantidad inicial es 0.

- F(1) = 1

Este es el segundo **caso base**, que establece que el **valor en la posición 1** es 1. Es decir, si queremos el "primer número real" de la sucesión, es 1.

Junto con F(0), estos dos valores son **los únicos que no se calculan**, sino que se **definen directamente**.

- F(n) = F(n-1) + F(n-2), para n ≥ 2

Esta es la **regla recursiva** o **relación de recurrencia**. Significa que **cualquier número en la sucesión a partir del índice 2** se calcula como la **suma de los dos anteriores**.

En otras palabras:

- Para obtener F(2), sumamos F(1) + F(0)
- Para obtener F(3), sumamos F(2) + F(1)
- Para obtener F(4), sumamos F(3) + F(2)
- Y así sucesivamente...

Esta parte es clave porque define la **lógica repetitiva o recursiva** del algoritmo. Cada número depende de los dos anteriores.

Esta característica convierte a la sucesión de Fibonacci en un ejemplo clásico para enseñar **recursividad**: una función que se llama a sí misma hasta alcanzar los casos base.

4.2 Resolviendo Fibonacci de forma recursiva

¿Qué es la recursividad?

La recursividad **es una forma de resolver problemas en la que una función se llama a sí misma.** Sirve cuando el problema se puede **dividir en versiones más pequeñas del mismo problema**.

Por ejemplo, **calcular el término n** de la secuencia de Fibonacci **requiere conocer los dos anteriores**: **fibonacci(n) = fibonacci(n - 1) + fibonacci(n - 2)**. Eso nos da una estructura naturalmente recursiva.

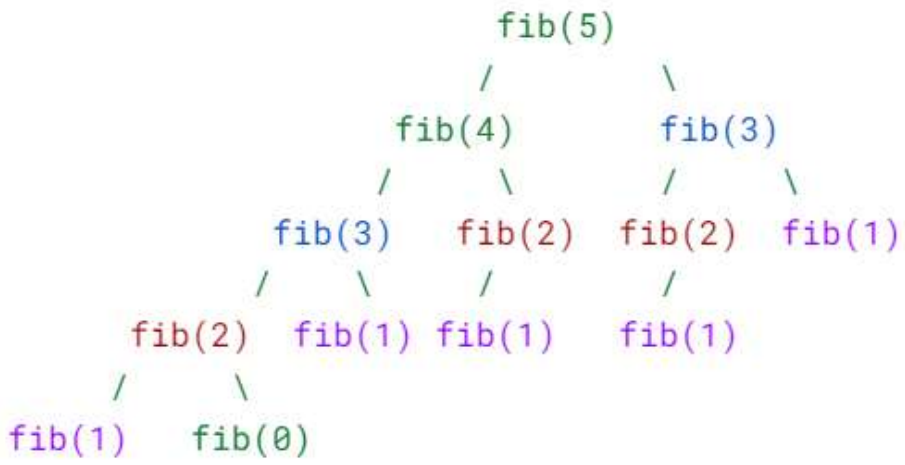
Ejemplo en Java:

```
int fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Análisis de este enfoque:

- Es **intuitivo** y fácil de implementar.
- Pero tiene un **gran problema de eficiencia**.

Veamos qué pasa cuando llamamos `fibonacci(5)`:



¡Muchos cálculos se repiten innecesariamente! Como se ve, fibonacci(2) se calcula varias veces. A medida que n crece, la cantidad de repeticiones se vuelve enorme.

4.3 Complejidad de la solución recursiva

Cada llamada genera **dos nuevas llamadas**. Esto genera una estructura de árbol binario de llamadas.

- La cantidad de llamadas crece **exponencialmente**.
- **Complejidad temporal:** $O(2^n)$

Ejemplo:

- Para `fibonacci(5)` → ~15 llamadas
- Para `fibonacci(40)` → más de **300 millones** de llamadas

Es un claro ejemplo de algoritmo mal diseñado si lo usamos sin optimización.

4.4 Alternativa: Programación Dinámica

¿Qué es programación dinámica?

Es una técnica de optimización que evita cálculos repetidos **almacenando los resultados ya calculados**. Esto se llama **memorización**. Hay dos formas de usar programación dinámica:

- **Top-down** (con recursión + memoización)
- **Bottom-up** (iterativa, como el ejemplo)

En este caso usamos la versión bottom-up, que es más eficiente en Java porque evita el uso de la pila de llamadas**. Implementación iterativa (bottom-up):**

```
int fibonacciDinamico(int n) {
    if (n == 0) return 0;
    int a = 0, b = 1;

    for (int i = 2; i <= n; i++) {
        int temp = a + b;
        a = b;
        b = temp;
    }
    return b;

}
```

Complejidad:

- **Tiempo:** O(n) ¡Pasamos de millones de llamadas a unas pocas operaciones!

4.6 Conclusión

El problema de Fibonacci nos permite ver cómo un algoritmo puede funcionar bien en teoría, pero ser **poco práctico** en la realidad si no lo diseñamos correctamente.

- La **solución recursiva** es buena para explicar conceptos, pero **no es eficiente**.
- La **programación dinámica** es una herramienta poderosa que permite transformar una solución ineficiente en una muy rápida.

Esta comparación muestra claramente por qué es fundamental **analizar la complejidad de un algoritmo antes de usarlo**, especialmente en casos reales donde el tamaño de entrada puede ser muy grande.

Aprender recursividad está bien para entender cómo se divide un problema, pero también es importante saber cuándo conviene usarla y cuándo no.

Por eso, al resolver problemas, **no alcanza con que el algoritmo funcione: también tiene que hacerlo de forma eficiente**.

¿Podés pensar otro problema que se pueda resolver recursivamente y que también tenga una versión optimizada con programación dinámica?

5. Estrategias de Resolución de Problemas

A lo largo del curso veremos diferentes tipos de problemas que pueden resolverse mediante distintos **enfoques algorítmicos**. Es importante saber que **no todos los problemas se resuelven igual**, y que existen **estrategias comunes** que se aplican según la naturaleza del problema.

Cuando enfrentamos un problema complejo, saber qué estrategia usar puede marcar la diferencia entre encontrar una solución rápidamente… o quedarnos atascados.

A lo largo del curso, vamos a incorporar varias de estas estrategias que los programadores usan todo el tiempo para resolver problemas reales.

A continuación, mencionamos algunas de las estrategias más utilizadas en algoritmos. No vamos a profundizar más en esta unidad, pero es importante que las conozcamos de nombre para poder identificarlas cuando aparezcan.

Principales estrategias:

- **Divide and Conquer (Dividir y conquistar):**Se basa en dividir un problema en subproblemas más pequeños, resolverlos de manera recursiva y luego combinar las soluciones. Es muy eficiente para ciertos tipos de problemas, como ordenamientos o búsqueda (Merge Sort, Binary Search, Quick Sort) Por ejemplo, buscar una palabra en un diccionario ordenado cortándolo por la mitad cada vez
- **Backtracking (Vuelta atrás):** Consiste en ir construyendo una solución paso a paso, y retroceder cuando se detecta que no se puede continuar. Es típico en problemas de exploración como laberintos, rompecabezas, o combinatorias. (Sudoku, N-Queens, resolver laberinto), por ejemplo, probar opciones y volver atrás si no funciona
- **Greedy Algorithms (Algoritmos codiciosos):** Esta estrategia toma decisiones locales óptimas en cada paso con la esperanza de llegar a una solución global óptima. Suele aplicarse en problemas de optimización, como seleccionar caminos más cortos o distribuir recursos. (Prim, Kruskal). por ejemplo, siempre elegir la opción que parece mejor en el momento.
- **Programación Dinámica:** Ya la utilizamos en el ejemplo de Fibonacci. Esta técnica optimiza problemas dividiéndolos en subproblemas y guardando los resultados para no repetir cálculos. (Fibonacci, mochila, caminos mínimos) por ejemplo, recordar soluciones anteriores para no repetir trabajo.

Estrategia	¿Cuándo se usa?
Divide and Conquer	Problemas que pueden dividirse recursivamente
Backtracking	Problemas de exploración o combinatorios
Greedy	Cuando se puede tomar decisiones óptimas locales
Programación Dinámica	Cuando hay subproblemas repetidos con superposición

Estas estrategias serán nuestras herramientas fundamentales para resolver muchos de los problemas que veremos. A medida que practiquemos, iremos reconociendo cuándo aplicar cada una.

¿Te animás a identificar qué estrategia usarías para resolver un Sudoku? ¿Y para buscar una palabra en un diccionario?

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.